CocoConnect

A Digital Ecosystem for Coconut Estate Management

Technical Feasibility Document

Client: Coconut Growers Association of Sri Lanka · Group 38

Prepared by: S.N. Vidanage (2023/IS/108)

Contents

Contents 2
1. Executive Summary 3
2. Purpose and Scope 3
3. System Overview 3
4. Technology Feasibility 3
5. Module Feasibility 3
6. External Integrations 4
7. Infrastructure and Cost 4
8. Security Feasibility 4
9. Performance and Scalability 5
10. Team Skills and Resources 5
11. Technical Risks 5
12. Constraints 5
13. Conclusion and Recommendations 6

1. Executive Summary

This study assesses whether CocoConnect can be built and run with the chosen technologies, the team's skills, the available time (two semesters) and a near-zero budget. Each module, external integration and infrastructure need was examined.

Conclusion: CocoConnect is technically feasible. All core modules can be built with Flutter, Node.js with Express.js, and PostgreSQL. Offline harvest recording and location features are feasible with extra care. Online payment processing is not feasible within the project's time and budget and is deferred to a later phase.

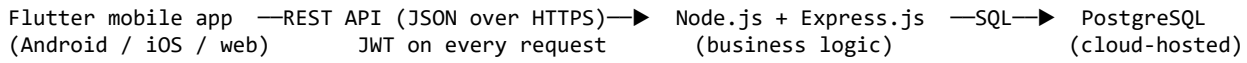
2. Purpose and Scope

The purpose of this document is to confirm, before full development, that the system described in the Software Requirements Specification (Version 3) can be delivered. It covers:

- the suitability of the technology stack;
- the feasibility of each functional module;
- external services such as maps, notifications and payments;
- hosting, cost, security and performance;
- team skills, time and the main technical risks.

3. System Overview

CocoConnect is a mobile-first platform for the coconut industry with a web portal for administration. It connects estate owners, estate managers, buyers, sellers, workers, service providers and administrators.



4. Technology Feasibility

Layer	Technology	Feasibility	Evaluation
Mobile frontend	Flutter (Dart)	**High**	One codebase for Android and iOS; mature widget library; built-in localisation for English, Sinhala and Tamil; hot reload speeds up development
Backend	Node.js + Express.js	**High**	Lightweight REST API framework; large package ecosystem; JavaScript is familiar to the team
Database	PostgreSQL	**High**	Relational model fits the linked data (users, estates, listings, orders); supports transactions and foreign keys; free and open source
Database hosting	Supabase (managed PostgreSQL)	**High**	Cloud-hosted PostgreSQL with a free tier and a web dashboard, so the whole team shares one database
Authentication	JWT + bcrypt	**High**	Standard, well-documented libraries; token-based login suits mobile apps; passwords stored as hashes
Maps and location	OpenStreetMap (flutter_map) + device GPS	**High**	Free map tiles and place search with no API key; GPS available on all target phones
API documentation	Swagger / OpenAPI	**High**	Generated from route comments; lets the team test routes before the app is connected
Version control	Git + GitHub	**High**	Free; branches let each member work on their own module

5. Module Feasibility

Module	Feasibility	Technical approach	Main challenge
User Management	**High**	Registration with role selection; bcrypt password hashing; JWT login; middleware that checks the token and role on every protected route	Keeping each role limited to its own functions

Module	Feasibility	Technical approach	Main challenge
Product Marketplace	**High**	Listings and buyer requests stored in PostgreSQL; filtered and sorted REST endpoints (type, price, date, quantity)	Matching requests to listings by quantity and date
Negotiation and orders	**High**	Chat messages with optional price offers; accepting an offer creates an order inside a database transaction	Keeping both sides consistent when an offer is accepted
Workforce Marketplace	**High**	Service profiles, hiring requests and job status, using the same patterns as the product marketplace	Several worker and service categories
Estate Management	**High**	Estate table linked to owner and manager; simple create, update and view screens	None significant
Harvest Management	**Medium**	Records saved on the phone when offline, marked with a sync status, and uploaded when a connection returns	Offline storage and conflict-free synchronisation
Location-based search	**Medium**	Store each user's coordinates; calculate distance on the server with the haversine formula; filter by radius	Accuracy of GPS in rural estates
Rating and Review	**High**	Rating and review records linked to completed orders or jobs	Allowing reviews only after a completed transaction
Notifications	**High**	In-app alerts built from database activity (offers, messages, requests); push notifications later	Real-time delivery without a push service
Administration	**High**	Admin-only API routes and a web portal over existing tables	None significant
Online Payment	**Low**	Would need a payment gateway, merchant registration and extra security checks	Cost, compliance and time; deferred

6. External Integrations

Integration	Option assessed	Result	Notes
Maps	Google Maps API	**Feasible with conditions**	High quality, but needs a billing account and API key, with charges after free usage
Maps	OpenStreetMap + Nominatim	**Feasible**	Free with no key; good coverage of Sri Lankan towns and estates; chosen
Push notifications	Firebase Cloud Messaging	**Feasible with conditions**	Free, but needs Firebase project setup per platform; in-app alerts first, push later
SMS / OTP	Local SMS gateway	**Feasible with conditions**	Charged per message; not needed for the first release
Payments	Online payment gateway	**Deferred**	Needs merchant registration and fees; payment stays outside the app (cheque or bank transfer) in the first release

7. Infrastructure and Cost

Item	Option	Cost during the project
Database	Supabase free tier	Free
Backend hosting	Local machine during development; free or low-cost cloud host for demonstration	Free to low
Maps	OpenStreetMap tiles and Nominatim	Free
Development tools	Flutter SDK, Node.js, VS Code, Android Studio, Git	Free
Source control	GitHub	Free
Mobile publishing	Google Play developer account (only if published)	One-time fee; not required for the project

The system can be built and demonstrated at almost no cost, which meets the project's budget constraint.

8. Security Feasibility

- Passwords are hashed with bcrypt; plain passwords are never stored.

- Every protected request carries a signed JWT, checked by middleware with the user's role.
- Secrets such as the database connection string are kept in environment files, never in the source code repository.
- Users' exact coordinates stay on the server; other users only see distances.
- Communication uses HTTPS in deployment.

All of these use standard, well-supported libraries, so the security requirements (NFR-04 to NFR-07) are feasible.

9. Performance and Scalability

- Response time under 3 seconds (NFR-01) is achievable: queries filter in the database and return only the needed rows.
- Node.js handles many concurrent requests without blocking (NFR-02).
- New products are added as categories in the data, not new code (NFR-15).
- The modular backend (separate routes and controllers per area) lets new modules be added without changing existing ones (NFR-16).
- Slow rural connections are handled with request timeouts, clear error messages, and offline harvest recording.

10. Team Skills and Resources

Area	Skill level	Plan to close gaps
Flutter / Dart	Medium	Official documentation and building small screens first
Node.js / Express.js	Medium	Start from simple routes; reuse one pattern for all modules
SQL / PostgreSQL	Medium to high	Builds on database coursework
UI/UX design	Medium	Figma wireframes reviewed with the client before coding
Testing	Medium	Automated API tests plus manual testing on the Android emulator
Hardware	Available	Team laptops can run the Android emulator; physical Android phones for field testing

11. Technical Risks

Risk	Likelihood	Impact	Mitigation
Weak GPS and internet in rural estates	High	High	Offline harvest recording with later sync; manual location pick on a map
Low digital literacy among users	High	Medium	Simple screens, three languages, large buttons and clear messages
Learning curve for Flutter and Node.js	Medium	Medium	Small prototypes first; shared code patterns across modules
Integrating separately built modules	Medium	High	Agreed API contracts in Swagger; one shared database; regular merges on GitHub
Free-tier limits of hosted services	Low	Medium	Enough for development and demonstration; paid plans only if deployed widely
Third-party service downtime	Medium	Medium	Keep core features independent of external services; fall back gracefully
Scope growing beyond the timeline	Medium	High	Prioritised requirements (Must/Should/Could); defer payments and analytics

12. Constraints

- Two-semester timeline with a student team.
- Near-zero budget: free tiers and open-source tools only.
- Must work on low-end Android phones and slow connections.
- Must support English, Sinhala and Tamil.

13. Conclusion and Recommendations

CocoConnect is technically feasible with the chosen stack, the team's skills and the available budget.

Recommendation	Reason
Proceed with Flutter, Node.js with Express.js, and PostgreSQL on Supabase	All rated high feasibility; free; familiar to the team
Use OpenStreetMap instead of Google Maps	Free, no billing account, adequate coverage
Build User Management and the Product Marketplace first	Every other module depends on users and listings
Build offline harvest recording as a separate, later step	Medium feasibility; needs local storage and sync
Defer online payments	Low feasibility within time and budget; payment stays outside the app
Document every API in Swagger and agree it before connecting the app	Reduces integration risk between team members